GUIA TÉCNICO

Fundamentos de bancos de dados

Modelos, persistência, critérios de escolha e limites dos testes locais

Propósito — Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

Como utilizar — Leia as explicações, execute os exemplos no ambiente indicado e consulte os notebooks correspondentes.

ROTEIRO DO DOCUMENTO

1. 01 | Fundamentos de bancos de dados
2. Objetivo e escopo
3. Conceito central: Critérios de escolha
4. Aplicação, ressalvas e limites
5. Dado, informação e persistência
6. Comparação dos modelos
7. Exercício resolvido - escolhas de arquitetura
8. Erros comuns

Fundamentos de bancos de dados

Modelos, persistência, critérios de escolha e limites dos testes locais

MAPA DE ESTUDO

01 Dados, informação e persistência

02 Comparação de modelos de dados

03 Critérios de escolha por carga de trabalho

04 Integridade, disponibilidade e custos

05 Exercícios e decisões arquiteturais

Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

01 | Fundamentos de bancos de dados

GUIA TÉCNICO APROFUNDADO • ANÁLISE, CÓDIGO E INTERPRETAÇÃO

Objetivo e escopo

Estudo independente baseado no notebook aprovado. Os exemplos locais usam SQLite e dados sintéticos; PostgreSQL, MongoDB e Cassandra requerem serviços de laboratório para verificação real.

Conceito central: Critérios de escolha

Diferencie modelo lógico e produto de banco de dados. Compare integridade, padrão de acesso, latência, concorrência e custo operacional antes de decidir. SQLite é excelente para prática local, mas não equivale a testar uma instância PostgreSQL sob concorrência.

Aplicação, ressalvas e limites

Dados de sensores: exija campos estáveis, identificadores e regra de validade. Em cenários de ingestão crescente, determine consultas típicas e política de retenção antes de escolher arquitetura.

Dado, informação e persistência

Dado é registro; informação é dado contextualizado. Um SGBD oferece armazenamento, consulta, controle de acesso e regras de integridade. Arquivos CSV são úteis, mas não garantem integridade referencial nem transações entre múltiplas tabelas.

Exemplo do notebook — execução local

```
df = pd.read_csv(DATA/'leituras_sensores.csv')
print(df.shape); display(df.head());print(df.dtypes)
```

Comparação dos modelos

Relacional: tabelas, esquema, relacionamentos e SQL. Documental: documentos aninhados e consultas por campo. Colunar distribuído: particionamento orientado ao padrão de acesso. Não existe banco universalmente melhor; escolha por consultas, consistência, escala e operação.

Exemplo do notebook — execução local

```
comparacao=pd.DataFrame({'modelo':['Relacional','Documental','Wide-column'],
    'exemplo':['PostgreSQL','MongoDB','Cassandra'],'forte_em':['integridade
    e SQL','documentos flexíveis','distribuição e consultas predefinidas']})
    ;display(comparacao)
```

Exercício resolvido - escolhas de arquitetura

Problema: histórico de sensores da casa inteligente com consultas frequentes por data e sensor. Solução justificada: começar pelo relacional devido a chaves e consultas analíticas; considerar soluções distribuídas somente após medir volume e requisitos de disponibilidade.

Exemplo do notebook — execução local

```
print('Decisão: SQLite para prototipar; PostgreSQL para concorrência e trans
```

```
    ações de servidor.')
```

Erros comuns

Confundir disponibilidade com ausência de erros, NoSQL com ausência de esquema, e escalabilidade com um requisito presente em qualquer projeto.

--- ### Aprofundamento v2 - Persistência, restrições e decisão tecnológica

A seleção de um SGBD depende de *consultas, integridade, volume, concorrência, disponibilidade e equipe*. Armazenar CSV em disco não equivale a controlar transações ou integridade referencial. Compare OLTP (operações curtas, concorrentes) e OLAP (agregações analíticas); a divisão é de carga de trabalho, não de fornecedor.

Interpretação: compare o resultado com a teoria anterior; este exemplo local não substitui um laboratório de serviço externo.

Exemplo do notebook — execução local

```
import sqlite3
with sqlite3.connect(':memory:') as db:
    db.execute('PRAGMA foreign_keys = ON')
    db.executescript('CREATE TABLE pessoas(id INTEGER PRIMARY KEY, nome TEXT
     NOT NULL); CREATE TABLE pedidos(id INTEGER PRIMARY KEY, pessoa_id INTEG
    ER NOT NULL REFERENCES pessoas(id));')
    db.execute('INSERT INTO pessoas VALUES (1, "Ana")')
    db.execute('INSERT INTO pedidos VALUES (1, 1)')
    try:
        db.execute('INSERT INTO pedidos VALUES (2, 999)')
    except sqlite3.IntegrityError:
        print('Integridade referencial protegida pelo banco.')
    else:
        raise AssertionError('Restrição FK não aplicada')
```

Exercício de revisão

Exercício resolvido - revisão: identifique a principal hipótese do exemplo acima, explique por que ela importa e cite uma condição na qual a técnica falharia. Resposta orientadora: registre a pré-condição (tipo de dado, chave, restrição, volume ou topologia), teste-a explicitamente e explique os limites dos resultados observados.

Aprofundamento: decisões e estudo de caso

Modelos de armazenamento e granularidade

Antes de escolher o mecanismo de persistência, identifique o evento que cada linha ou documento representa. No caso dos sensores, a unidade é uma leitura de um dispositivo em determinado instante. O identificador do evento precisa ser único e a relação entre sensor, cômodo e medida deve ser explícita. Esse modelo evita que a contagem de registros seja confundida com a contagem de sensores.

Integridade e concorrência

Uma restrição PRIMARY KEY garante identificação; FOREIGN KEY verifica correspondência entre entidades; CHECK e NOT NULL delimitam domínios. Entretanto, essas restrições só demonstram sua eficácia se forem efetivamente habilitadas e testadas. O exemplo local do SQLite protege uma FK: é um teste real da funcionalidade

local, mas não reproduz os desafios de transações concorrentes de vários clientes.

Plano de consultas e custo

Uma aplicação que lê a última temperatura de cada sensor precisa consultar por identificador e tempo. A frequência de escrita, a retenção e o crescimento podem tornar inadequada uma varredura integral. Se a consulta principal muda para dashboards por região, agrupamentos e junções se tornam relevantes. A tecnologia deve ser avaliada pela carga representativa, não apenas pela popularidade.

Exercício aplicado

Escreva três perguntas concretas de consulta para os dados de sensores. Para cada uma, registre: campos de filtro, ordenação, cardinalidade esperada, exigência de atomicidade e eventual necessidade de auditoria. Depois compare como a mesma pergunta seria atendida em uma tabela relacional, documento aninhado e partição wide-column. Justifique ao menos um custo e uma limitação de cada proposta.

Checklist de interpretação

• Defina a pergunta, unidade de análise e pré-condições da consulta.

• Registre chaves, cardinalidade, restrições e tratamento de valores ausentes.

• Execute os exemplos locais antes de habilitar serviços reais.

• Separe evidência observada de hipóteses e propriedades não testadas.

Leituras e arquivos de apoio

Consulte o notebook correspondente, os scripts em sql/, os datasets sintéticos e a documentação oficial dos SGBDs. Os resultados locais não substituem testes de integração com serviços externos.